

Chapter 1

Intro to approximation algorithms

Sven-Olai Høyland

Chapter 1

- Discrete optimization
Decision variables (or some of them) are restricted to finite or discrete values such as integers
- NP-hard problems
 - Informally: "at least as hard as the hardest problems in NP"
- Most interesting discrete optimization problems are NP-hard
 - What should we do?

Approach to solve a NP-hard problem

- Ideally
 - Find optimal solution
 - In polynomial time
 - For any instance
- But we have to relax at least one of the requirements to deal with a NP-hard problem
 - Can choose any two of the three above

Most common

- Instead of optimal solution, settle for a solution that is good enough
 - Use various types of heuristics and metaheuristics
 - [Simulated annealing](#)
 - [Genetic algorithms](#)
 - [Tabu search](#)
 - Approximation algorithms (main topic in this course)

Approximation algorithms

- Relax the requirement of finding an optimal solution
- Try to find a solution that closely approximates the optimal solution in term of its value
- Objective function maps each possible solution to some nonnegative value
- Optimal solution is one that either minimizes or maximizes the value of the objective function

Approximation algorithms

- More formally
- Definiton 1.1: An α -approximation algorithm for an optimization problem is a **polynomial-time** algorithm that for all instances of the problem produces a solution whose value is within **a factor α** of the value of an optimal solution
- We call α the performance guarantee of the algorithm (also called approximation ratio or approximation factor)

Performance guarantee

- Our book
 - $\alpha > 1$ for minimization problems
 - $\alpha < 1$ for maximization problems
- Example
 - An $\frac{1}{2}$ -approximation algorithm for a maximization problems is a polynomial-time algorithm that always returns a value whose value is at least half the optimal value
- Other books may define $\alpha > 1$ also for maximization problems (approximationRatio = $1/\alpha$)

Why approximation algorithms?

- Because we need algorithms to get solutions to discrete optimization problems
- Because algorithm design often focuses first on idealized models rather than the «real-world» application
- Because it provides a mathematically rigorous basis on which to study heuristics
- Because it gives a metric for stating how hard various discrete optimization problems are
- Because it is fun 😊

Poly-time approximation schemes

- Definition 1.2
 - A polynomial-time approximation scheme (PTAS) is a family of algorithms $\{A_\varepsilon\}$ where there is an algorithm for each $\varepsilon > 0$, such that A_ε is a $(1+\varepsilon)$ -approximation algorithm (for minimization problems) or a $(1-\varepsilon)$ -approximation (for a maximization problems)
 - Smaller $\varepsilon \rightarrow$ longer time
 - Examples: Knapsack and Euclidean travelling salesperson

Class MAX SNP (Strict NP)

- This class contains problems where there are very difficult to find approximation algorithms
- Theorem 1.3
For any MAX SNP-hard problem, there does not exist a polynomial-time approximation scheme unless $P = NP$
- Let n denote the number of vertices in an input graph, and consider any constant $\varepsilon > 0$. Then there does not exist an $O(n^{\varepsilon-1})$ -approximation algorithm for the maximum clique problem unless $P = NP$.

Background

- Introduce
 - The set cover problem
 - Will use it to show different techniques for approximation algorithms
 - Linear programming (LP)

Set cover problem

- Ground set of elements $E = \{e_1, e_2, \dots, e_n\}$
- Subsets of the elements $S_1, S_2, \dots, S_m$ where $S_j \subseteq E$
- Each subset has a nonnegative weight $w_j \geq 0$
- Goal: Find a minimum weight collection of subsets that covers all of E . That is find a subset I of $\{1, 2, \dots, m\}$ that minimizes $\sum_{j \in I} w_j$ subject to $\bigcup_{j \in I} S_j = E$
- If $w_j = 1$ for each subset j , the problem is called the **unweighted** set cover problem

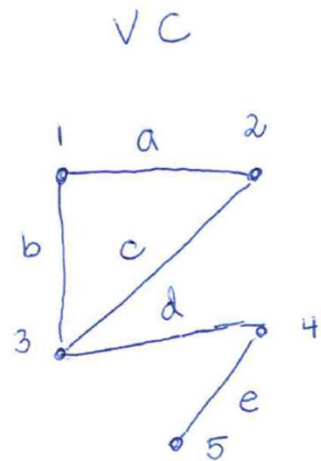
Example

- $E = \{1,2,3,4\}$
- Subsets
 - $S_1 = \{1,2\}, w_1 = 2$
 - $S_2 = \{1,3\}, w_2 = 1$
 - $S_3 = \{2,4\}, w_3 = 2$
 - $S_4 = \{3,4\}, w_4 = 5$
- Feasible solutions?
- Optimal solution?

Set cover problem

- Abstraction of several types of problems
 - Used in the development of an antivirus product
 - Vertex cover problem (Norwegian: nodedekke)
 - How to formulate Vertex cover in terms of Set cover
 - Ground set $\leftrightarrow$ Edges
 - Subsets $\leftrightarrow$ One for each vertex containing all incident edges
 - Now a set cover will correspond to a vertex cover
 - Example next slide

Vertex cover $\rightarrow$ Set cover



SC

$$E = \{a, b, c, d, e\}$$

$$S_1 = \{a, b\}$$

$$S_2 = \{a, c\}$$

$$S_3 = \{b, c, d\}$$

$$S_4 = \{d, e\}$$

$$S_5 = \{e\}$$

Linear programming (LP)

- Plays a central role in design and analysis of approximation algorithms
- Important terms
 - **Decision variables**
represent some sort of decisions that need to be made
 - **Constraints**
Variables are constrained by a number of linear inequalities
 - **Feasible solutions**
Any assignment of real numbers to the variables such that the constraints are satisfied
- Exists efficient algorithms to solve LP

LP, Example

- minimize $2x_1 + x_2$
- Subject to $x_1 + 3x_2 \geq 4$
 $3x_1 + 2x_2 \geq 12$

} Constraints

$$x_i \geq 0$$

- Decision variables: x_1 and x_2
- Example of a feasible solution (both constraints are satisfied): $x_1 = 3$ and $x_2 = 2$
- Infeasible solution (first constraints is satisfied, but not the second): $x_1 = 1$ and $x_2 = 1$

Integer programming

- Integer linear programming (ILP)
 - Exclude fractional solutions
- Zero-one linear programming
 - Allow only 0 or 1 as values for decision variables
- Integer programming is NP-hard

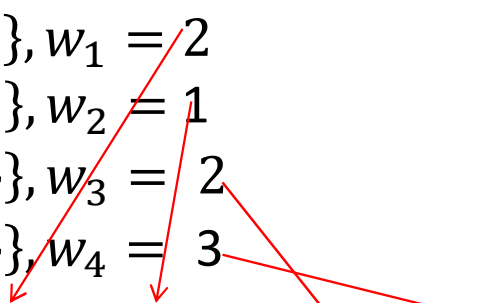
Set cover as ILP

n elements

m subsets

$$\begin{array}{ll}\text{minimize} & \sum_{j=1}^m w_j x_j \\ \text{subject to} & \sum_{j: e_i \in S_j} x_j \geq 1, \quad i = 1, \dots, n \\ & x_j \in \{0,1\}, \quad j = 1, \dots, m\end{array}$$

Example

- $S = \{1,2,3,4\}$
 - Subsets
 - $S_1 = \{1,2\}, w_1 = 2$
 - $S_2 = \{1,3\}, w_2 = 1$
 - $S_3 = \{2,4\}, w_3 = 2$
 - $S_4 = \{3,4\}, w_4 = 3$
 - minimize $2x_1 + 1x_2 + 2x_3 + 3x_4$
 - subject to «all elements must be present in at least one of the subsets choosen», see next slide for details
 - $x_j \in \{0,1\}, j = 1, \dots, 4$
- 

Constraints from previous slide in detail

$$x_1 + x_2 \geq 1 \text{ (means that 1 is covered)}$$

$$x_1 + x_3 \geq 1 \text{ (means that 2 is covered)}$$

$$x_2 + x_4 \geq 1 \text{ (means that 3 is covered)}$$

$$x_3 + x_4 \geq 1 \text{ (means that 4 is covered)}$$

Filled in during lecture

x_1	x_2	x_3	x_4	Feasible (y/n)	value
0	0	0	0		
0	0	0	1		
0	0	1	0		
0	0	1	1		
0	1	0	0		
0	1	0	1		
0	1	1	0		
0	1	1	1		
1	0	0	0		
1	0	0	1		
1	0	1	0		
1	0	1	1		
1	1	0	0		
1	1	0	1		
1	1	1	0		
1	1	1	1		

Filled in

x_1	x_2	x_3	x_4	Feasible (y/n)	value
0	0	0	0	no	
0	0	0	1	no	
0	0	1	0	no	
0	0	1	1	no	
0	1	0	0	no	
0	1	0	1	no	
0	1	1	0	yes	3
0	1	1	1	yes	6
1	0	0	0	no	
1	0	0	1	yes	5
1	0	1	0	no	
1	0	1	1	yes	7
1	1	0	0	no	
1	1	0	1	yes	6
1	1	1	0	yes	5
1	1	1	1	yes	8



Optimal value

Comments

- For small instances as in previous example, we can enumerate all possible combinations for values on decision variables (2^n), check if the solution is feasible and find the best feasible solution
- A lot of real problems can be formulated as ILPs
- Even if ILP is NP-complete, if the problems are not too large we have software that can solve them efficient.

Set cover

ILP

$$\begin{aligned} &\text{minimize} && \sum_{j=1}^m w_j x_j \\ &\text{subject to} && \sum_{j: e_i \in S_j} x_j \geq 1, \quad i = 1, \dots, n \\ & && x_j \in \{0,1\}, \quad j = 1, \dots, m \end{aligned}$$

Notation: in $j: e_i \in S_j$ the $:$ means such that

LP relaxation

$$\begin{aligned} &\text{minimize} && \sum_{j=1}^m w_j x_j \\ &\text{subject to} && \sum_{j: e_i \in S_j} x_j \geq 1, \quad i = 1, \dots, n \\ & && x_j \geq 0, \quad j = 1, \dots, m \end{aligned}$$

Notice, very important

- The LP version is a relaxation of the ILP since
 - Every feasible solution to ILP is still a solution to LP
 - Any feasible solution to ILP has the same value in LP
- Let Z_{LP}^* denote the optimum value of the linear program
- Any optimal solution to the ILP is feasible for the LP
- Thus $Z_{LP}^* \leq Z_{ILP}^* = \text{OPT}$

Plan

- Show examples of how the LP relaxation can be used to derive approximation algorithms

Deterministic rounding algorithm

- Solve LP relaxation
Let x^* denote an optimal solution to LP
- Include subset S_j in our solution if and only if $x_j \geq 1/f$ where f is the maximum number of sets in which any element appears
 - *Example ($f = 3$ since element 2 is in 3 sets)*

$$\begin{aligned} S_1 &= \{1, 2\} \\ S_2 &= \{1, 3\} \\ S_3 &= \{2, 4\} \\ S_4 &= \{2, 3, 4\} \end{aligned}$$

Deterministic rounding algorithm (2)

- Lemma 1.5. The collection of subsets $S_j, j \in I$, is set cover
 - Proof: Know that $\sum_{j: e_i \in S_j} x_j \geq 1$, for every element. There are at most f terms in the sum. Hence at least one of the terms is $\geq 1/f$ and will be rounded to 1.

Performance guarantee

- Theorem 1.6. The rounding algorithm is an f -approximation algorithm for the set cover problem
- Proof
 - Poly time is ok
 - We have: $1 \leq f \cdot x_j^*$ for each $j \in I$ and since $f w_j x_j^*$ is nonnegative we have:
 - $\sum_{j \in I} w_j \leq \sum_{j=1}^m w_j \cdot (f \cdot x_j^*)$

$$\begin{aligned} &= f \sum_{j=1}^m w_j \cdot x_j^* = \\ &f Z_{LP}^* \leq f \cdot OPT \end{aligned}$$

Rounding a dual solution

- Each element e_i is charged a nonnegative price $y_i \geq 0$ for its coverage by a set cover
- Reasonable prices $\sum_{i:e_i \in S_j} y_i \leq w_j$
- Highest total prices the elements can be charged
 - maximize $\sum_{i=1}^n y_i$
 - subject to $\sum_{i:e_i \in S_j} y_i \leq w_j, \quad j = 1, \dots, m,$
 $y_i \geq 0, \quad i = 1, \dots, n$
- Formulation above is called the dual linear program of the set cover problem
- The first LP formulation is called the primal LP
- Given a LP we can always formulate the dual LP

Important properties

- Any feasible solution to the dual LP has a value not greater than any feasible solution to the primal LP (primal LP is a minimization problem)
 - $\sum_{i=1}^n y_i \leq \sum_{j=1}^m w_j x_j$ (weak duality property)
- Strong duality property of linear programs
 - As long there exist feasible solution to both the primal and the dual LP, their optimal value are equal
 - If x^* is an optimal solution to the set cover LP relaxation and y^* is an optimal solution to the dual LP, then $\sum_{j=1}^m w_j x_j^* = \sum_{i=1}^n y_i^*$

Primal-Dual method (1)

- LPs are efficiently solvable and algorithms to solve them are quick in practice



Special purpose algorithms are often much faster

Primal-Dual method (2)

- $\sum_{i=1}^n y_i \leq OPT$ (true for any feasible dual solution)
- Include $j \in I'$ precisely when $\sum_{i: e_i \in S_j} y_i = w_j$



Cost of I' is no more than f times optimal

Primal-dual algorithm

$y \leftarrow 0$

$I \leftarrow \emptyset$

while there exists $e_i \notin \bigcup_{j \in I} S_j$ **do**

 Increase the dual variable y_i until there is some ℓ with $e_i \in S_\ell$ such that $\sum_{j: e_j \in S_\ell} y_j = w_\ell$

$I \leftarrow I \cup \{\ell\}$

Example

- $S = \{1,2,3,4\}$
- Subsets
 - $S_1 = \{1,2\}, w_1 = 2$
 - $S_2 = \{1,3\}, w_2 = 1$
 - $S_3 = \{2,4\}, w_3 = 2$
 - $S_4 = \{3,4\}, w_4 = 3$
- minimize $2x_1 + 1x_2 + 2x_3 + 3x_4$
- subject to «all elements must be present in at least one of the subsets chosen»
- $x_j \in \{0,1\}, j = 1, \dots, 4$
- Dual?


$$\begin{array}{rclcl} 1x_1 & + & 1x_2 & \geq & 1 \\ 1x_1 & + & 1x_3 & \geq & 1 \\ 1x_2 & + & 1x_4 & \geq & 1 \\ 1x_3 & + & 1x_4 & \geq & 1 \end{array}$$

Dual

- Maximize $y_1 + y_2 + y_3 + y_4$
Subject to

$$y_1 + y_2 \leq 2$$

$$y_1 + y_3 \leq 1$$

$$y_2 + y_4 \leq 2$$

$$y_3 + y_4 \leq 3$$

$$y_i \geq 0$$

One solution

- $y=(0,0,0,0)$, $I = \emptyset$
- Choose y_1 , Can be increased to 1 (inequality 2)
 $y=(1,0,0,0)$, $I = \{2\}$
- Choose y_2 , Can be increased to 1 (inequality 1)
 $y=(1,1,0,0)$, $I = \{1,2\}$
- Choose y_4 , Can be increased to 1 (inequality 3)
 $y=(1,1,0,1)$, $I = \{1,2,3\}$
- Done
- Start by choosing y_4 first

Performance of the Primal-dual algorithm

- Theorem 1.9. The Primal-dual algorithm is an f -approximation algorithm for the set cover problem

Greedy algorithm

- Make a sequence of decisions
 - Each is made to optimize that decision
 - Local optimal, but might not be global optimal
- Typical very easy to implement

Greedy algorithm

$$I \leftarrow \emptyset$$

$$\hat{S}_j \leftarrow S_j \quad \forall j$$

while I is not a set cover **do**

$$\ell = \arg \min_{j: \hat{S}_j \neq \emptyset} \frac{w_j}{|\hat{S}_j|}$$

$$I \leftarrow I \cup \{\ell\}$$

$$\hat{S}_j \leftarrow \hat{S}_j - S_\ell \quad \forall j$$

Example

- $S = \{1,2,3,4\}$
- Subsets
 - $S_1 = \{1,2\}, w_1 = 2$
 - $S_2 = \{1,3\}, w_2 = 1$
 - $S_3 = \{2,4\}, w_3 = 2$
 - $S_4 = \{3,4\}, w_4 = 3$
- minimize $2x_1 + 1x_2 + 2x_3 + 3x_4$
- subject to «all elements must be present in at least one of the subsets chosen»
- $x_j \in \{0,1\}, j = 1, \dots, 4$

Performance of the the greedy algortihm

- Fact 1.10. Given positive numbers $a_1, \dots, a_k$ and $b_1, \dots, b_k$, then

$$\min_{i=1,\dots,k} \frac{a_i}{b_i} \leq \frac{\sum_{i=1}^k a_i}{\sum_{i=1}^k b_i} \leq \max_{i=1,\dots,k} \frac{a_i}{b_i}$$

- Theorem 1.11. The greedy algorithm given is an H_n -approximation algorithm for the set cover problem, where $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$
(n is the number of elements in the ground set)

Improved guarantee

- By using the dual formulation, it is possible to achieve a better guarantee (same algorithm) $g = \max_j |S_j|$
 - Theorem 1.12. Let $g = \max_j |S_j|$. The greedy algorithm returns a solution indexed by I such that $\sum_{j \in I} w_j \leq H_g \cdot Z^*_{LP}$
 - Why is this better?

possible that $g = n$, but that will mean we have a subset covering the groundset

Some negative results (1)

- Theorem 1.13. If there exists a $c \ln n$ -approximation algorithm for the unweighted set cover problem for some constant $c < 1$, then there is an $O(n^{\log 10})$ -time deterministic algorithm for each NP-complete problem.
- Theorem 1.14. There exists some constant $c > 0$ such that if there exists a $c \ln n$ -approximation algorithm for the unweighted set cover problem, then $P = NP$.

Some negative results (2)

- Theorem 1.15. If there exists an α -approximation algorithm for the vertex cover problem with a $\alpha < 10\sqrt{5} - 21 \approx 1.36$, then $P = NP$.
- Theorem 1.16. Assuming the unique games conjecture holds, if there exists an α -approximation algorithm for the vertex cover problem with constant $\alpha < 2$, then $P = NP$.

Following slides

- Contain some result that we will come back to later in the course
- Do not panic if it looks strange

A Randomized rounding algorithm

- Solve the LP relaxation
- Round the fractional solution to an integral solution
- Instead deterministic round, we do it randomly (called randomized rounding)
- Let x^* be an optimal solution to the LP relaxation. We would like to round fractional values of x^* to either 0 or 1 and obtain a solution to the set cover problem without increasing the cost to much

Randomized rounding (2)

- Let $\hat{x}_j$ be value of x_j obtained by algorithm
- Central idea
We consider x_j^* as the probability that $\hat{x}_j$ should be set to 1
- Quite likely that we will not have a set cover
- By doing this $c \ln n$ times for each variable, we will have an algorithm the fails with probability at most $\frac{1}{n^c}$

Randomized rounding (3)

- Theorem 1.17. The algorithm is a randomized $O(\ln n)$ -approximation algorithm that produces a set cover with high probability
- We have seen simpler and faster approximation algorithms that achieves a better performance guarantee
 - However, for some problems this approach works well (Chapter 5)